LCS Memoization To Tabulation

Yes. **Uncrossed Lines (1035)** is essentially the **Longest Common Subsequence (LCS)** problem.

The important part is understanding **why the recursion works**, and then how that exact recursion becomes the **2D tabulation**.

---

1. First understand the problem

We have:

```
nums1 = [1, 4, 2]
nums2 = [1, 2, 4]
```

We can connect equal numbers.

If we connect:

```
1 → 1
```

then we can connect another pair only if their order is preserved.

For example:

```
nums1: 1   4   2
       |       |
nums2: 1   2   4
```

We can choose:

```
1 → 1
2 → 2
```

giving **2**.

But:

```
4 → 4
2 → 2
```

would cross because the order is reversed.

Key observation

We need to find:

> **The longest sequence of values that appears in both arrays in the same order.**

That is exactly **LCS**.

---

2. DP state

Let's define:

```
C++
dp[i][j]
```

as:

> Maximum number of uncrossed lines we can make using
> **`nums1[i ... n-1]`** and **`nums2[j ... m-1]`**.

So:

```
dp(i, j)
       |
       +---- nums1[i]
       |
       +---- nums2[j]
```

We compare:

```
C++
nums1[i] == nums2[j]
```

There are two cases.

3. Case 1: Numbers are equal

Suppose:

```
nums1[i] = 2
nums2[j] = 2
```

We should connect them.

So we get:

```
1 line
```

and move both pointers:

```
i → i + 1
j → j + 1
```

Therefore:

```
C++
dp[i][j] = 1 + dp[i+1][j+1];
```

Why both move?

Because once we connect these two numbers, neither number can be used again.

---

4. Case 2: Numbers are different

Suppose:

```
nums1[i] = 4
nums2[j] = 2
```

We cannot connect them.

So we have two choices:

Choice 1

Skip **`nums1[i]`**:

```
C++
dp[i+1][j]
```

Choice 2

Skip **`nums2[j]`**:

```
C++
dp[i][j+1]
```

Take the better one:

```
C++
dp[i][j] = max(dp[i+1][j], dp[i][j+1]);
```

5. Recursion tree idea

The complete recurrence is:

```
C++
if (nums1[i] == nums2[j])
    return 1 + solve(i+1, j+1);

else
    return max(
        solve(i+1, j),
        solve(i, j+1)
    );
```

Think of it like this:

```
                solve(i,j)
                    |
```

```
                    |
          ┌─────────┴──────────┐
          |                    |
      equal?                different
          |                    |
          ↓                    ↓
   1 + solve(i+1,j+1)    ┌─────┴──────┐
                         |            |
                    skip nums1    skip nums2
                         |            |
                    solve(i+1,j)   solve(i,j+1)
```

6. Memoization code

```
C++
class Solution {
public:

    int solve(vector<int>& nums1,
              vector<int>& nums2,
              int i,
              int j,
              vector<vector<int>>& memo) {

        // No elements left
        if (i == nums1.size() || j == nums2.size())
            return 0;

        // Already calculated
        if (memo[i][j] != -1)
            return memo[i][j];

        // If numbers match
        if (nums1[i] == nums2[j]) {
            return memo[i][j] =
                1 + solve(nums1, nums2, i + 1, j + 1, memo);
        }

        // If numbers don't match
        return memo[i][j] =
            max(
                solve(nums1, nums2, i + 1, j, memo),
                solve(nums1, nums2, i, j + 1, memo)
            );
    }

    int maxUncrossedLines(vector<int>& nums1,
                          vector<int>& nums2) {

        int n = nums1.size();
        int m = nums2.size();

        vector<vector<int>> memo(n, vector<int>(m, -1));

        return solve(nums1, nums2, 0, 0, memo);
    }
};
```

7. Now the important part: converting Memoization → Tabulation

Your recursion says:

```
C++
solve(i, j)
```

depends on:

```
C++
solve(i+1, j+1)
solve(i+1, j)
solve(i, j+1)
```

Look carefully.

```
              solve(i,j)
             /    |    \
            ↓     ↓     ↓
       (i+1,j+1) (i+1,j) (i,j+1)
```

Therefore, when doing bottom-up DP, we need those states **already calculated**.

That's why we go:

```
i = n-1 → 0
j = m-1 → 0
```

instead of:

```
0 → n-1
0 → m-1
```

8. Add the extra row and column

This makes tabulation much easier.

Instead of:

```
C++
dp[n][m]
```

we create:

```
C++
dp[n+1][m+1]
```

Why?

Because our recursion has:

```
C++
if (i == n || j == m)
    return 0;
```

Those boundary states can simply be represented by:

```
dp[n][j] = 0
dp[i][m] = 0
```

So:

```
          nums2
        1   2   4   END
  1   |   |   |   | 0 |
  4   |   |   |   | 0 |
  2   |   |   |   | 0 |
 END  | 0 | 0 | 0 | 0 |
```

The last row and last column are automatically `0`.

---

9. Convert each recursive statement

Memoization:

```
C++
solve(i+1, j+1)
```

becomes:

```
C++
dp[i+1][j+1]
```

Memoization:

```
C++
solve(i+1, j)
```

becomes:

```
C++
dp[i+1][j]
```

Memoization:

```
C++
solve(i, j+1)
```

becomes:

```
C++
dp[i][j+1]
```

Therefore:

```
C++
if (nums1[i] == nums2[j]) {
    dp[i][j] = 1 + dp[i+1][j+1];
}
else {
    dp[i][j] = max(dp[i+1][j], dp[i][j+1]);
}
```

That's literally the recursion converted to bottom-up.

---

10. Tabulation code

```
C++
class Solution {
public:

    int maxUncrossedLines(vector<int>& nums1,
                          vector<int>& nums2) {

        int n = nums1.size();
        int m = nums2.size();

        vector<vector<int>> dp(n + 1,
                               vector<int>(m + 1, 0));

        // Start from bottom-right
        for (int i = n - 1; i >= 0; i--) {

            for (int j = m - 1; j >= 0; j--) {

                if (nums1[i] == nums2[j]) {

                    dp[i][j] = 1 + dp[i + 1][j + 1];

                } else {

                    dp[i][j] = max(
                        dp[i + 1][j],
                        dp[i][j + 1]
                    );
                }
            }
        }

        return dp[0][0];
    }
};
```

11. Let's actually build the table

Take:

```
nums1 = [1,4,2]
nums2 = [1,2,4]
```

Initial table:

```
       1   2   4   END
    ┌───┬───┬───┬─────┐
1   │ 0 │ 0 │ 0 │  0  │
4   │ 0 │ 0 │ 0 │  0  │
2   │ 0 │ 0 │ 0 │  0  │
END │ 0 │ 0 │ 0 │  0  │
    └───┴───┴───┴─────┘
```

We start from:

```
i = 2
j = 2
```

Step 1

```
nums1[2] = 2
nums2[2] = 4
```

Not equal.

So:

```
C++
dp[2][2] = max(dp[3][2], dp[2][3])
         = max(0, 0)
         = 0;
```

Table:

```
      1   2   4
   ┌───┬───┬───┐
1  │ 0 │ 0 │ 0 │
4  │ 0 │ 0 │ 0 │
2  │ 0 │ 0 │ 0 │ ←
   └───┴───┴───┘
```

Step 2

Now:

```
i = 2
j = 1

nums1[2] = 2
nums2[1] = 2
```

Equal!

Therefore:

```
C++
dp[2][1] = 1 + dp[3][2]
         = 1 + 0
         = 1;
```

```
      1   2   4
   ┌───┬───┬───┐
1  │ 0 │ 0 │ 0 │
4  │ 0 │ 0 │ 0 │
2  │ 0 │ 1 │ 0 │
   └───┴───┴───┘
```

We found one line:

```
2 → 2
```

Step 3

```
i = 2
j = 0

nums1[2] = 2
nums2[0] = 1
```

Different.

So:

```
C++
dp[2][0] = max(
    dp[3][0],
    dp[2][1]
);
```

```
= max(0, 1)
= 1
```

Table:

```
      1   2   4
   ┌───┬───┬───┐
1  │ 0 │ 0 │ 0 │
4  │ 0 │ 0 │ 0 │
2  │ 1 │ 1 │ 0 │
   └───┴───┴───┘
```

12. Next row: `nums1[i] = 4`

Start:

```
i = 1
```

j = 2

```
4 vs 4
```

Match:

```
C++
dp[1][2] = 1 + dp[2][3]
         = 1
```

Now:

```
       1   2   4
   ┌───┬───┬───┐
1  │ 0 │ 0 │ 0 │
4  │ 0 │ 0 │ 1 │
2  │ 1 │ 1 │ 0 │
   └───┴───┴───┘
```

j = 1

```
4 vs 2
```

Different:

```
C++
dp[1][1] = max(
    dp[2][1],
    dp[1][2]
);
```

Therefore:

```
max(1,1) = 1
```

```
       1   2   4
   ┌───┬───┬───┐
1  │ 0 │ 0 │ 0 │
4  │ 0 │ 1 │ 1 │
2  │ 1 │ 1 │ 0 │
   └───┴───┴───┘
```

j = 0

```
4 vs 1
```

Different:

```
C++
dp[1][0] = max(
    dp[2][0],
    dp[1][1]
);
```

```
= max(1,1)
= 1
```

Table:

```
       1   2   4
   ┌───┬───┬───┐
1  │ 0 │ 0 │ 0 │
4  │ 1 │ 1 │ 1 │
2  │ 1 │ 1 │ 0 │
   └───┴───┴───┘
```

13. Finally: row for 1

```
i = 0
```

j = 2

```
1 vs 4
```

Different:

```
dp[0][2] = max(dp[1][2], dp[0][3])
         = max(1,0)
         = 1
```

j = 1

```
1 vs 2
```

Different:

```
dp[0][1] = max(dp[1][1], dp[0][2])
         = max(1,1)
         = 1
```

j = 0

```
1 vs 1
```

Match!

```
C++
dp[0][0] = 1 + dp[1][1]
         = 1 + 1
         = 2;
```

Final table:

```
          nums2
          1   2   4
nums1 1 | 2 | 1 | 1 |
     4  | 1 | 1 | 1 |
     2  | 1 | 1 | 0 |

            ↑
          answer
```

So:

```
C++
dp[0][0] = 2
```

14. The most important pattern to remember

This problem is a perfect example of converting **top-down DP → bottom-up DP**.

Memoization

You think in terms of:

```
What happens from (i,j)?
```

and recursion goes:

```
(i,j)
 ↓
(i+1,j+1)
(i+1,j)
(i,j+1)
```

Tabulation

You reverse that dependency.

Because:

```
dp[i][j]
needs
dp[i+1][j+1]
dp[i+1][j]
dp[i][j+1]
```

we calculate:

```
bottom → top
right → left
```

So:

```
C++
for (int i = n - 1; i >= 0; i--) {
    for (int j = m - 1; j >= 0; j--) {
```

And the exact recurrence remains:

```
C++
if (nums1[i] == nums2[j])
    dp[i][j] = 1 + dp[i+1][j+1];
else
    dp[i][j] = max(dp[i+1][j], dp[i][j+1]);
```

That's the whole conversion.

The recursion and tabulation are not two different logics. **Tabulation is just executing the same recursive recurrence in the correct dependency order.**

Complexity

For `n = nums1.size()` and `m = nums2.size()`:

```
Memoization:
Time  = O(n × m)
Space = O(n × m) + recursion stack

Tabulation:
Time  = O(n × m)
Space = O(n × m)
```

And the biggest connection to remember is:

> **Uncrossed Lines = LCS = same state + same recurrence + different execution order.**